UNIT- 5

PYTHON PROGRAMMING

Errors and Exceptions:

As human beings, we commit several errors. A software developer is also a human being and hence prone to commit errors wither in the design of the software or in writing the code. The errors in the software are called „bugs" and the process of removing them are called „debugging". In general, we can classify errors in a program into one of these three types:

a) Compile-time errors
b) Runtime errors
c) Logical errors

a) Compile-time errors

These are syntactical errors found in the code, due to which a program fails to compile. For example, forgetting a colon in the statements like if, while, for, def, etc. will result in compile-time error. Such errors are detected by python compiler and the line number along with error description is displayed by the python compiler.

Example: A Python program to understand the compile-time error.

```
a = 1
if a == 1
   print “hello”
```

Output:

```
File ex.py, line 3
  If a == 1
          ^
SyntaxError: invalid syntax
```

b) Runtime errors

When PVM cannot execute the byte code, it flags runtime error. For example, insufficient memory to store something or inability of PVM to execute some statement come under runtime errors. Runtime errors are not detected by the python compiler. They are detected by the PVM, Only at runtime.

Example: A Python program to understand the compile-time error.

```
print "hai"+25
```

Output:

```
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    print "hai"+25
TypeError: cannot concatenate 'str' and 'int' objects
```

c) Logical errors

These errors depict flaws in the logic of the program. The programmer might be using a wrong formula of the design of the program itself is wrong. Logical errors are not detected either by Python compiler of PVM. The programme is solely responsible for them. In the following program, the programmer wants to calculate incremented salary of an employee, but he gets wrong output, since he uses wrong formula.

Example: A Python program to increment the salary of an employee by 15%.

```
def increment(sal):
    sal = sal * 15/100
    return sal
sal = increment(5000)
print "Salary after Increment is", sal
```

Output:

```
Salary after Increment is 750
```

From the above program the formula for salary is wrong, because only the increment but it is not adding it to the original salary. So, the correct formula would be:

```
sal = sal + sal * 15/100
```

Compile time errors and runtime errors can be eliminated by the programmer by modifying the program source code.

In case of runtime errors, when the programmer knows which type of error occurs, he must handle them using exception handling mechanism.

Exceptions:

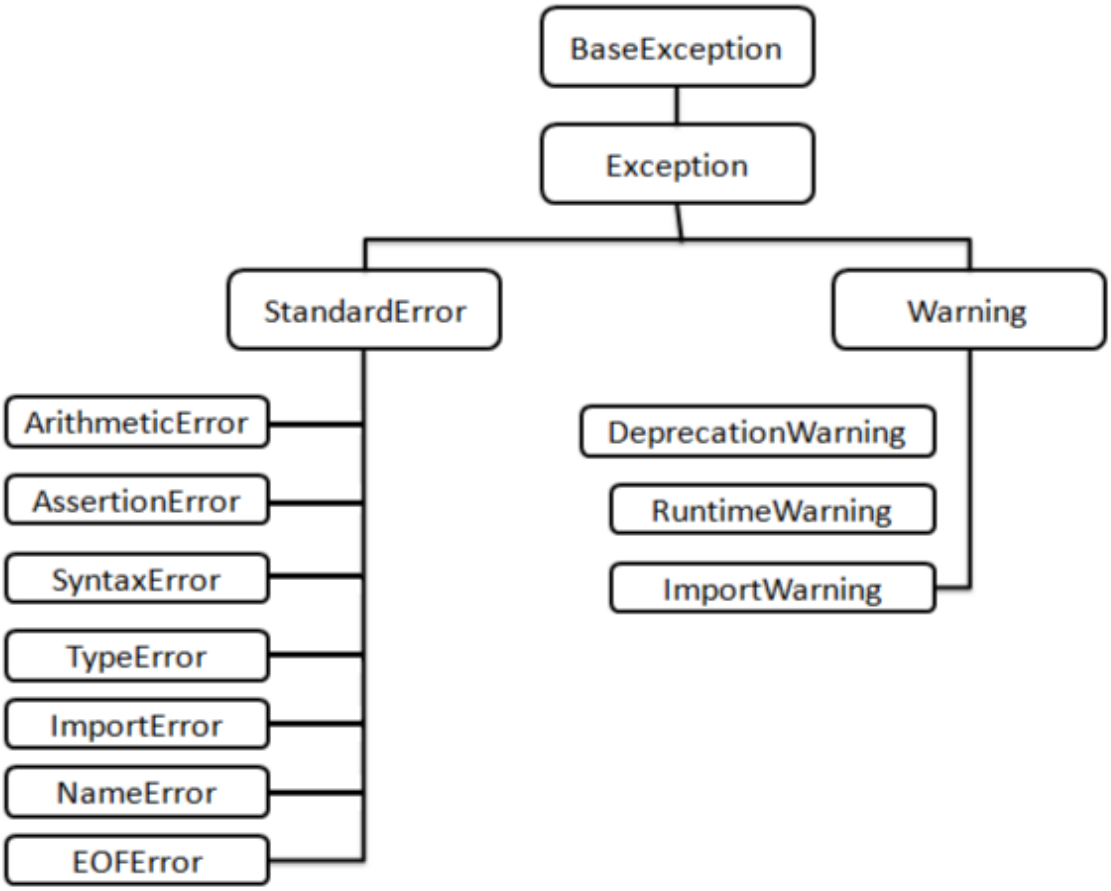
- An exception is a runtime error which can be handled by the programmer.
- That means if the programmer can guess an error in the program and he can do something to eliminate the harm caused by that error, then it is called an „exception".
- If the programmer cannot do anything in case of an error, then it is called an „error" and not an exception.
- All exceptions are represented as classes in python. The exceptions which are already available in python are called „built-in" exceptions. The base class for all built-in exceptions is „BaseException" class.
- From BaseException class, the sub class „Exception" is derived. From Exception class, the sub classes „StandardError" and „Warning" are derived.

- ✓ All errors (or exceptions) are defined as sub classes of StandardError. An error should be compulsory handled otherwise the program will not execute.

- ✓ Similarly, all warnings are derived as sub classes from „Warning" class. A warning represents a caution and even though it is not handled, the program will execute. So, warnings can be neglected but errors cannot neglect.

- ✓ Just like the exceptions which are already available in python language, a programmer can also create his own exceptions, called „user-defined" exceptions.

- ✓ When the programmer wants to create his own exception class, he should derive his class from Exception class and not from "BaseException" class.



Exception Handling:

- ✓ The purpose of handling errors is to make the program *robust*. The word "robust" means "strong". A robust program does not terminate in the middle.

- ✓ Also, when there is an error in the program, it will display an appropriate message to the user and continue execution.

- ✓ Designing such programs is needed in any software development.

- ✓ For that purpose, the programmer should handle the errors. When the errors can be handled, they are called exceptions.

- ✓ To handle exceptions, the programmer should perform the following four steps:

- ✓ **Step 1:** The programmer should observe the statements in his program where there may be a possibility of exceptions. Such statements should be written inside a „try" block. A try block looks like as follows:

try:
statements

The greatness of try block is that even if some exception arises inside it, the program will not be terminated. When PVM understands that there is an exception, it jumps into an „except" block.

Step 2: The programmer should write the „except" block where he should display the exception details to the user. This helps the user to understand that there is some error in the program. The programmer should also display a message regarding what can be done to avoid this error. Except block looks like as follows:

except exceptionname:
statements

The statements written inside an except block are called „handlers" since they handle the situation when the exception occurs.

Step 3: If no exception is raised, the statements inside the „else" block is executed. Else block looks like as follows:

else:
statements

Step 4: Lastly, the programmer should perform clean up actions like closing the files and terminating any other processes which are running. The programmer should write this code in the finally block. Finally block looks like as follows:

finally:
statements

The speciality of finally block is that the statements inside the finally block are executed irrespective of whether there is an exception or not. This ensures that all the opened files are properly closed and all the running processes are properly terminated. So, the data in the files will not be corrupted and the user is at the safe-side.

Here, the complete exception handling syntax will be in the following format:

```
try:
    statements
except Exception1:
    statements
except Exception2:
    statements
else:
    statements
finally:
    statements
```

The following points are followed in exception handling:

- ✓ A single try block can be followed by several except blocks.
- ✓ Multiple except blocks can be used to handle multiple exceptions.
- ✓ We cannot write except blocks without a try block.
- ✓ We can write a try block without any except blocks.
- ✓ Else block and finally blocks are not compulsory.
- ✓ When there is no exception, else block is executed after try block.
- ✓ Finally block is always executed.

Example: A python program to handle IOError produced by open() function.

```
import sys
try:
   f = open('myfile.txt','r')
   s = f.readline()
   print s
   f.close()
except IOError as e:
   print "I/O error", e.strerror
except:
   print "Unexpected error:"
```

Output:

```
I/O error No such file or directory
```

In the if the file is not found, then IOError is raised. Then „except‟ block will display a message: „I/O error‟. if the file is found, then all the lines of the file are read using readline() method.

List of Standard Exceptions:-

Exception Name	Description
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.

EnvironmentError	Base class for all exceptions that occur outside the Python environment.
IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
OSError	Raised for operating system-related errors.
SyntaxError	Raised when there is an error in Python syntax.
IndentationError	Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

The Except Block:

The “except“ block is useful to catch an exception that is raised in the try block. When there is an exception in the try block, then only the except block is executed. it is written in various formats.

1. To catch the exception which is raised in the try block, we can write except block with the Exceptionclass name as:
   except Exceptionclass:
2. We can catch the exception as an object that contains some description about the exception.
   except Exceptionclass as obj:
3. To catch multiple exceptions, we can write multiple catch blocks. The other way is to use a single except block and write all the exceptions as a tuple inside parentheses as:
   except (Exceptionclass1, Exceptionclass2, ...):
4. To catch any type of exception where we are not bothered about which type of exception it is, we can write except block without mentioning any Exceptionclass name as:
   except:

Example:

```
try:
   f = open('myfile.txt','w')
   a=input("Enter a value ")
   b=input("Enter a value ")
   c=a/float(b)
   s = f.write(str(c))
   print “Result is stored”
except ZeroDivisionError:
   print "Division is not possible"
except:
   print "Unexpected error:"
finally:
   f.close()
```

Output:

```
Enter a value 1
Enter a value 5
Result is stored
```

Raising an Exception: -

You can raise exceptions in several ways by using the raise statement. The general syntax for the **raise** statement is as follows.

raise [Exception [, args [, traceback]]]

Here, *Exception* is the type of exception (For example, NameError) and *argument* is a value for the exception argument. The argument is optional; if not supplied, the exception argument is None.

For Example, If you need to determine whether an exception was raised but don‟t intend to handle it, a simpler form of the raise statement allows you to re-raise the exception:

```
try:
   raise NameError('HiThere')
except NameError:
   print 'An exception flew by!'
   raise
```

User-Defined Exceptions:

- ✓ Like the built-in exceptions of python, the programmer can also create his own exceptions which are called „User-defined exceptions‟ or „Custom exceptions‟. We know Python offers many exceptions which will raise in different contexts.

- ✓ But, there may be some situations where none of the exceptions in Python are useful for the programmer. In that case, the programme has to create his/her own exception and raise it.

- ✓ For example, let‟s take a bank where customers have accounts. Each account is characterized should by customer name and balance amount.

- ✓ The rule of the bank is that every customer should keep minimum Rs. 2000.00 as balance amount in his account.

- ✓ The programmer now is given a task to check the accounts to know every customer is maintaining minimum balance of Rs. 2000.00 or not.

- ✓ If the balance amount is below Rs. 2000.00, then the programmer wants to raise an exception saying „Balance amount is less in the account of so and so person.“ This will be helpful to the bank authorities to find out the customer.

- ✓ So, the programmer wants an exception that is raised when the balance amount in an account is less than Rs. 2000.00. Since there is no such exception available in python, the programme has to create his/her own exception.

For this purpose, he/she has to follow these steps:

1. Since all exceptions are classes, the programme is supposed to create his own exception as a class. Also, he should make his class as a sub class to the in-built
   “Exception“ class.

   ```
   class MyException(Exception):
   def __init__(self, arg):
       self.msg = arg
   ```

   Here, MyException class is the sub class for „Exception“ class. This class has a constructor where a variable „msg“ is defined. This „msg“ receives a message passed from outside through „arg“.
2. The programmer can write his code; maybe it represents a group of statements or a function. When the programmer suspects the possibility of exception, he should raise his own exception using „raise“ statement as:

   ```
   raise MyException('message')
   ```

   Here, raise statement is raising MyException class object that contains the given
   “message“.
3. The programmer can insert the code inside a „try“ block and catch the exception using “except“ block as:

   ```
   try:
   code
   except MyException as
      me: print me
   ```

   Here, the object „me“ contains the message given in the raise statement.

All these steps are shown in below program.

Example:

```
class MyException(Exception):
   def __init__(self, arg):
      self.msg = arg
def check(dict):
   for k,v in dict.items():
      print "Name=",k,"Balance=",v
      if v<2000.00:
         raise MyException("Balance amount is less in the
         account of "+k)

bank={"ravi":5000.00,"ramu":8500.00,"ra
ju":1990.00} try:
   check(bank)
except MyException as me:
   print me.msg
```

Output:

```
Name= ramu Balance= 8500.0
Name= ravi Balance= 5000.0
Name= raju Balance= 1990.0
Balance amount is less in the account of raju
```